



School of Computing

**M33147-INTELLIGENT DATA AND TEXT
ANALYTICS**

Coursework #

Date of Submission: 19-01-2026

Module Lecturers: Dr Atefeh Khazaei

Student ID: UP2302123

UP2312756

UP2301714

Group Member	Main Responsibilities	Specific Tasks Completed	Approx. % Contribution
UP2312756	Preprocessing	Cleaned dataset, lemmatization, stopwords	30%
UP2301714	ML models	BoW, NB, LR, SVM, evaluation	30%
UP2302123	Deep learning & report	BERT, LDA, writing	40%

Contents

Introduction.....	4
Dataset Description	5
Task 1: Text Preprocessing	6
Task 1.1 Removal of Punctuation	7
Task 1.2 Removal of Numbers.....	7
Task 1.3 Conversion to Lowercase.....	7
Task 1.4 Stopword Removal	7
Task 1.5 Lemmatization	7
Task 2: Feature Extraction and Classification using Bag of Words	9
Task 2.1 Bag of Words Feature Extraction	9
Task 2.2 Train–Test Split	9
Task 2.3 Naive Bayes Classification.....	10
Task 2.4 Logistic Regression Classification	10
Task 2.5 Support Vector Machine (SVM) Classification	11
Task 2.6 Model Evaluation	11
Task 3: Sentiment Classification using BERT	12

Task 3.1 BERT Model Overview	12
Task 3.2 Tokenization and Dataset Preparation	12
Task 3.3 Model Training	12
Task 3.4 Model Evaluation	13
Task 3.5 Comparison with Traditional Models	14
Task 4: Topic Detection using Latent Dirichlet Allocation (LDA)	15
Task 4.1 Topic Modelling Overview	15
Task 4.2 Feature Extraction for Topic Modelling	15
Task 4.3 LDA Model Implementation	16
Task 4.4 Topic Interpretation	17
Task 4.5 Summary of Topic Detection	18
Conclusion	19
References	20

Introduction

Sentiment analysis is a subfield of Natural Language Processing (NLP) that focuses on identifying and classifying opinions expressed in text as positive or negative. With the rapid growth of online platforms, large volumes of user generated text such as movie reviews, product reviews and social media posts are generated every day. Analyzing this data helps organisations understand public opinion and make informed decisions.

In this coursework, sentiment analysis is performed in movie reviews from the IMDB labelled dataset. The objective is to classify movie review into positive and negative categories using different text analytics and machine learning techniques. The coursework explores the complete sentiment analysis pipeline starting from text preprocessing followed by feature extraction, classification using traditional machine learning models and finally deep learning using BERT. Additionally, topic modelling is applied to discover hidden themes within the dataset.

The aim of this coursework is to compare the performance of traditional machine learning models with a deep learning based approach and to analyse how different techniques affect sentiment classification accuracy.

Dataset Description

The dataset used in this coursework is the IMDB Labelled Sentences dataset which is commonly used for sentiment analysis tasks. The dataset consists of short movie review sentences collected from the Internet Movie Database (IMDB). Each sentence is labelled with a sentiment value where 1 represents a positive review and 0 represents a negative review.

The dataset contains approximately 1000 movie review sentences making it suitable for experimenting with different text analytics and machine learning techniques. The reviews are concise and cover a wide range of opinions related to movie quality, acting, storyline and overall viewer experience. The dataset is relatively balanced with a similar number of positive and negative examples.

Each record in the dataset consists of two columns:

- **Sentence:** A short movie review expressed in natural language.
- **Label:** A binary value indicating sentiment (1 for positive, 0 for negative).

This dataset was chosen because it provides a clean and well labelled benchmark for sentiment classification experiments. It allows the evaluation of both traditional machine learning models and deep learning based approaches such as BERT. The dataset was further processed through text preprocessing techniques before being used for model training and evaluation.

In addition, the compact size of the dataset makes it suitable for experimentation and rapid model evaluation. Despite its relatively small scale, the dataset contains diverse opinions expressed in natural language, which helps test the robustness of different sentiment analysis approaches. Using this dataset also allows clear observation of how preprocessing and feature extraction techniques influence classification performance.

Task 1: Text Preprocessing

Text Preprocessing is a crucial step in sentiment analysis because raw textual data often contains noise that can negatively affect the performance of machine learning models. Movie reviews usually include unnecessary characters, stopwords and different word forms which need to be cleaned and standardized before analysis.

In this task the IMDB labelled dataset was preprocessed using Python libraries such as NLT K and spacy. The aim of preprocessing was to convert the raw text into a clean and meaningful format suitable for feature extraction and classification.

```
You can now load the package via spacy.load('en_core_web_sm')
PS C:\Users\bharg_svlem99\OneDrive\Desktop\idta_new> python preprocessing.py
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\bharg_svlem99\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\stopwords.zip.
Preprocessing Finished
PS C:\Users\bharg_svlem99\OneDrive\Desktop\idta_new> █
```

The preprocessing steps significantly reduced noise in the dataset and improved the consistency of textual data. By standardising the reviews, the models were able to focus on sentiment-bearing words rather than irrelevant variations in the text. This step played a crucial role in improving the effectiveness of both traditional machine learning models and the deep learning approach applied in later tasks.

```
clean,label
slowmove aimless movie distressed drift young man,0
sure lose flat character audience nearly half walk,0
attempt artiness black white clever camera angle movie disappoint become even ridiculous act poor plot line almost nonexistent,0
little music anything speak,0
good scene movie gerardo try find song keep run head,1
rest movie lack art charm mean emptiness work guess empty,0
waste two hour,0
see movie today think good effort good message kid,1
bit predictable,0
love cast jimmy buffet science teacher,1
baby owl adorable,1
movie show lot florida well make look appeal,1
song good muppet hilarious,1
cool,1
```

Figure 1: Execution of preprocessing script using NLTK and spaCy.

Task 1.1 Removal of Punctuation

Punctuation marks such as commas, full stops, exclamation marks and question marks were removed from the text. These symbols do not provide useful information for sentiment classification and can increase the complexity of the feature space.

Task 1.2 Removal of Numbers

All numerical values were removed from the text. Numbers generally do not carry sentiment-related meaning in movie reviews and may introduce unnecessary noise into the dataset.

```
A very, very, very slow-moving, aimless movie about a distressed, drifting young man. 0
Not sure who was more lost - the flat characters or the audience, nearly half of whom walked out. 0
Attempting artiness with black & white and clever camera angles, the movie disappointed - became even more ridiculous - as
almost non-existent. 0
Very little music or anything to speak of. 0
The best scene in the movie was when Gerardo is trying to find a song that keeps running through his head. 1
The rest of the movie lacks art, charm, meaning... If it's about emptiness, it works I guess because it's empty. 0
Wasted two hours. 0
Saw the movie today and thought it was a good effort, good messages for kids. 1
A bit predictable. 0
Loved the casting of Jimmy Buffet as the science teacher. 1
And those baby owls were adorable. 1
The movie showed a lot of Florida at it's best, made it look very appealing. 1
The Songs Were The Best And The Muppets Were So Hilarious. 1
It Was So Cool. 1
```

Figure 2: Example of raw IMDB movie review sentences before preprocessing.

Task 1.3 Conversion to Lowercase

The entire text was converted to lowercase to maintain consistency across the dataset. This ensures that words such as Movie and movie are treated as the same token during analysis.

Task 1.4 Stopword Removal

Commonly used words such as the, is, and, and were removed using the NLTK stopwords list. These words occur frequently but do not contribute meaningful information for determining sentiment.

Task 1.5 Lemmatization

Lemmatization was applied using the spacy library to convert words into their base form. For example, running is converted to run and better to good. This helps reduce vocabulary size and improves model performance by grouping similar words together.

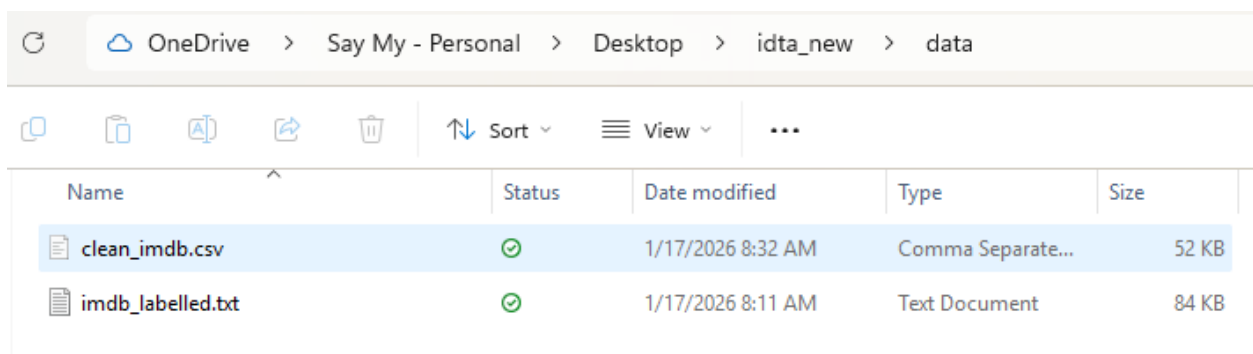
```

clean,label
slowmove aimless movie distressed drift young man,0
sure lose flat character audience nearly half walk,0
attempt artiness black white clever camera angle movie disappoint become even ridiculous act poor plot line almost nonexistent,0
little music anything speak,0
good scene movie gerardo try find song keep run head,1
rest movie lack art charm mean emptiness work guess empty,0
waste two hour,0
see movie today think good effort good message kid,1
bit predictable,0
love cast jimmy buffet science teacher,1
baby owl adorable,1
movie show lot florida well make look appeal,1
song good muppet hilarious,1
cool,1

```

Figure 3: Sample output of cleaned and lemmatized movie reviews.

After applying all preprocessing steps, the cleaned dataset was saved as `clean_imdb.csv`. This processed dataset was then used for classification and topic modelling tasks in later stages of the coursework.



The screenshot shows a OneDrive file explorer window with the path: OneDrive > Say My - Personal > Desktop > idta_new > data. The file list contains two items:

Name	Status	Date modified	Type	Size
 clean_imdb.csv		1/17/2026 8:32 AM	Comma Separate...	52 KB
 imdb_labelled.txt		1/17/2026 8:11 AM	Text Document	84 KB

Figure 4: Final preprocessed dataset saved as `clean_imdb.csv`.

Task 2: Feature Extraction and Classification using Bag of Words

After preprocessing the dataset in Task 1, the next step was to convert the cleaned text into a numerical representation that could be used by machine learning algorithms. In this task, the Bag of Words (BoW) technique was used for feature extraction followed by classification using different traditional machine learning models.

Bag of Words represents text by counting the frequency of words in each review, ignoring grammar and word order. This approach is simple yet effective for many text classification tasks.

Task 2.1 Bag of Words Feature Extraction

The Bag of Words model was implemented using the CountVectorizer from the scikit-learn library. Each movie review was transformed into a vector where each element represents the frequency of a particular word in the review.

```
bow_classification.py > ...
1  import pandas as pd
2  from sklearn.model_selection import train_test_split
3  from sklearn.feature_extraction.text import CountVectorizer
4  from sklearn.naive_bayes import MultinomialNB
5  from sklearn.linear_model import LogisticRegression
6  from sklearn.svm import LinearSVC
7  from sklearn.metrics import accuracy_score, classification_report
```

The cleaned dataset (clean_imdb.csv) was used as input, and the text data was converted into a numerical feature matrix. This matrix was then used for training and testing the classification models.

```
vectorizer = CountVectorizer()
X_bow = vectorizer.fit_transform(X)
```

Figure 2.1: Bag-of-Words feature extraction using CountVectorizer

Task 2.2 Train–Test Split

The dataset was divided into training and testing sets using an 80:20 split. The training set was used to train the models, while the testing set was used to evaluate their performance on unseen data. This approach helps assess how well the models generalise to new reviews.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_bow, y, test_size=0.2, random_state=42
)
```

Figure 2.2: Train–test split of the dataset using 80:20 ratio

Task 2.3 Naive Bayes Classification

The Multinomial Naïve Bayes classifier was applied to the Bag of Words features. Naive Bayes is a probabilistic classifier that works well with text data due to its assumption of feature independence. The model achieved reasonable accuracy in classifying positive and negative movie reviews.

```
=====
Naive Bayes
=====
Accuracy: 0.72
```

	precision	recall	f1-score	support
0	0.74	0.72	0.73	78
1	0.70	0.72	0.71	72
accuracy			0.72	150
macro avg	0.72	0.72	0.72	150
weighted avg	0.72	0.72	0.72	150

Figure 2.3: Performance of the Naive Bayes classifier on the IMDB dataset

Task 2.4 Logistic Regression Classification

Logistic Regression was used as another classification model. It is a linear classifier that estimates the probability of a review belonging to a particular sentiment class. Logistic Regression performed comparably to Naïve Bayes and provided stable and consistent results.

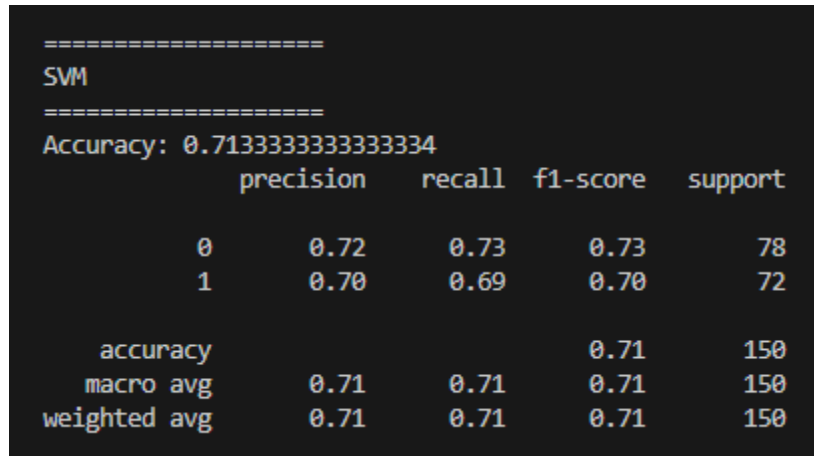
```
=====
Logistic Regression
=====
Accuracy: 0.72
```

	precision	recall	f1-score	support
0	0.75	0.69	0.72	78
1	0.69	0.75	0.72	72

Figure 2.4: Logistic Regression classification results

Task 2.5 Support Vector Machine (SVM) Classification

A Support Vector Machine (SVM) with a linear kernel was implemented for sentiment classification. SVM aims to find an optimal hyperplane that separates positive and negative reviews. This model also achieved competitive performance on the Bag of Words features.

A terminal window with a dark background and light-colored text. It displays the results of an SVM classification on the IMDB dataset. The output includes a title 'SVM', an overall accuracy of 0.7133333333333334, and a table of confusion matrix metrics for classes 0 and 1. Below the table, it shows the overall accuracy, macro average, and weighted average, all with a support of 150.

	precision	recall	f1-score	support
0	0.72	0.73	0.73	78
1	0.70	0.69	0.70	72
accuracy			0.71	150
macro avg	0.71	0.71	0.71	150
weighted avg	0.71	0.71	0.71	150

Figure 2.5: SVM classification performance on the IMDB dataset

Task 2.6 Model Evaluation

The performance of all three classifiers was evaluated using standard metrics including accuracy, precision, recall and F1-score. These metrics provided a detailed understanding of each models classification performance. Among the traditional machine learning models, Naïve Bayes and Logistic Regression achieved slightly higher accuracy compared to SVM.

The results demonstrate that Bag of Words combined with traditional classifiers can effectively perform sentiment classification; however these models have limitations in capturing contextual and semantic information from text.

Although the performance of the traditional classifiers was reasonable, their limitations became apparent when dealing with complex sentence structures. Since the Bag of Words approach ignores word order and semantic relationships, it struggles with expressions involving negation or implicit sentiment. These limitations motivated the exploration of a more advanced deep learning-based approach in the next task.

Task 3: Sentiment Classification using BERT

In this task, a deep learning-based approach was implemented for sentiment classification using BERT (Bidirectional Encoder Representations from Transformers). Unlike traditional machine learning models, BERT is capable of understanding the contextual meaning of words by analysing text bidirectionally. This allows the model to capture semantic relationships more effectively. The same preprocessed dataset used in Task 2 was utilised to ensure a fair comparison between traditional machine learning models and the deep learning approach.

Task 3.1 BERT Model Overview

The pretrained Bert-base-uncased model was used in this task. This model has been trained on a large corpus of English text and provides powerful language representations. The model was fine-tuned for binary sentiment classification, where reviews were classified as either positive or negative.

Task 3.2 Tokenization and Dataset Preparation

The cleaned movie reviews were tokenized using the BERT tokenizer. Tokenization converts text into numerical tokens that the BERT model can process. Padding and truncation were applied to ensure uniform input length. The dataset was then divided into training and testing sets using an 80:20 split.

```
1 import numpy as np
2 from sklearn.metrics import accuracy_score
3 import pandas as pd
4 from datasets import Dataset
5 from transformers import BertTokenizer, BertForSequenceClassification, TrainingArguments, Trainer
6 from sklearn.model_selection import train_test_split
7
```

Figure 3.1: Tokenization and dataset preparation using BERT tokenizer

Task 3.3 Model Training

The BERT model was fine-tuned on the training dataset for two epochs. During training the model learned to adapt its pretrained knowledge to the sentiment classification task. Training was performed using the HuggingFace Trainer API which simplifies the training and evaluation process.

```

training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    logging_strategy="epoch",
    num_train_epochs=2,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    save_strategy="no"
)

```

Figure 3.2: BERT training configuration using HuggingFace TrainingArguments

Task 3.4 Model Evaluation

After training, the BERT model was evaluated on the test dataset. Model performance was measured using accuracy which represents the proportion of correctly classified reviews. The BERT-based classifier achieved an accuracy of approximately 86.67%, which was significantly higher than the traditional Bag of Words models used in Task 2.

```

PS C:\Users\bharg_svlem99\OneDrive\Desktop\idta_new> python bert_classification.py
Map: 100%|██████████| 596/596 [00:00<00:00, 2434.24 examples/s]
Map: 100%|██████████| 150/150 [00:00<00:00, 3936.98 examples/s]
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at
sifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions
0%|██████████| 0/150 [00:00<?, ?it/s]C:\Users\bharg_svlem99\AppData
atolader.py:668: UserWarning: 'pin_memory' argument is set as true but no accelerator is found
warnings.warn(warn_msg)
{'loss': 0.5471, 'grad_norm': 4.500699996948242, 'learning_rate': 2.533333333333337e-05, 'epoch
Map: 100%|██████████| 150/150 [00:00<00:00, 3917.54 examples/s]
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at
sifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions
0%|██████████| 0/150 [00:00<?, ?it/s]C:\Users\bharg_svlem99\AppData
atolader.py:668: UserWarning: 'pin_memory' argument is set as true but no accelerator is found
warnings.warn(warn_msg)
{'loss': 0.5719, 'grad_norm': 2.989924669265747, 'learning_rate': 2.533333333333337e-05, 'epoch
{'eval_loss': 0.3863464295864105, 'eval_runtime': 20.8501, 'eval_samples_per_second': 7.194, 'e
{'loss': 0.2384, 'grad_norm': 1.249732255935669, 'learning_rate': 3.333333333333335e-07, 'epoch
{'eval_loss': 0.5018128752708435, 'eval_runtime': 21.4728, 'eval_samples_per_second': 6.986, 'e
{'train_runtime': 2762.6048, 'train_samples_per_second': 0.431, 'train_steps_per_second': 0.054
100%|██████████| 150/150 [46:02<00:00, 18.42s/it]
100%|██████████| 19/19 [00:20<00:00, 1.06s/it]
BERT Accuracy: 0.8666666666666667

```

Figure 3.3: Training and evaluation output of the BERT sentiment classification model

Task 3.5 Comparison with Traditional Models

The results demonstrate that BERT outperformed Naïve Bayes, Logistic Regression, and SVM models. This improvement is due to BERT's ability to capture contextual and semantic information, whereas Bag of Words models rely only on word frequency and ignore word order and context.

Task 4: Topic Detection using Latent Dirichlet Allocation (LDA)

In this task, topic detection was performed to identify hidden themes present in the movie reviews. Unlike sentiment classification, topic modelling is an unsupervised learning technique, meaning it does not use labelled data. The goal of this task was to explore the underlying topics discussed in the IMDb movie reviews after preprocessing.

Task 4.1 Topic Modelling Overview

Topic modelling aims to automatically discover abstract topics that occur in a collection of documents. In this coursework, Latent Dirichlet Allocation (LDA) was used for topic detection. LDA assumes that each document is a mixture of topics and that each topic is represented by a distribution of words.

Task 4.2 Feature Extraction for Topic Modelling

The cleaned dataset (`clean_imdb.csv`) generated in Task 1 was used as input for topic modelling. The text data was converted into a Bag of Words representation using `CountVectorizer`. This representation was required to provide numerical input to the LDA algorithm.

Stopwords were removed and low-frequency words were filtered to improve topic quality.

```

1  import pandas as pd
2  from sklearn.feature_extraction.text import CountVectorizer
3  from sklearn.decomposition import LatentDirichletAllocation
4
5  # Load cleaned dataset
6  df = pd.read_csv("data/clean_imdb.csv")
7
8  # Ensure no empty text
9  df = df.dropna(subset=["clean"])
10
11 texts = df["clean"].astype(str)
12
13 # Convert to BoW
14 vectorizer = CountVectorizer(stop_words="english")
15 X = vectorizer.fit_transform(texts)
16
17 # LDA
18 lda = LatentDirichletAllocation(n_components=10, random_state=42)
19 lda.fit(X)
20
21 words = vectorizer.get_feature_names_out()
22
23 for i, topic in enumerate(lda.components_):
24     print("\nTopic", i+1)
25     print([words[j] for j in topic.argsort()[-10:]])
26

```

Figure 4.1: Bag of Words feature extraction for LDA topic modelling

Task 4.3 LDA Model Implementation

The LDA model was implemented using the scikit-learn library. The number of topics was set to 10, as required by the coursework specification. The model was trained on the Bag of Words feature matrix to learn word distributions for each topic.

After training, the top ten most important words for each topic were extracted and displayed.

```
lda = LatentDirichletAllocation(n_components=10, random_state=42)
lda.fit(X)

words = vectorizer.get_feature_names_out()

for i, topic in enumerate(lda.components_):
    print("\nTopic", i+1)
    print([words[j] for j in topic.argsort()[-10:]])
```

Figure 4.2: Implementation of LDA model with 10 topics

Task 4.4 Topic Interpretation

The extracted topics revealed meaningful patterns within the movie reviews. The topics included words related to movie quality, storyline, acting, viewer experience and overall opinions about films. Although the topics were not explicitly labelled and the sets of keywords provided insight into the main themes discussed by reviewers.

Since LDA is an unsupervised technique, evaluation was performed qualitatively by analysing the coherence and interpretability of the extracted topics rather than using accuracy-based metrics.

```
Topic 1
['truly', 'single', 'highly', 'movie', 'play', 'make', 'way', 'great', 'recommend', 'film']

Topic 2
['brilliant', 'bad', 'know', 'place', 'like', 'avoid', 'character', 'actor', 'film', 'good']

Topic 3
['long', 'game', 'people', 'good', 'totally', 'watch', 'time', 'line', 'film', 'movie']

Topic 4
['way', 'character', 'hear', 'different', 'thing', 'scene', 'movie', 'make', 'plot', 'music']

Topic 5
['look', 'love', 'act', 'bad', 'really', 'funny', 'awful', 'movie', 'film', 'good']

Topic 6
['play', 'wonderful', 'waste', 'act', 'movie', 'like', 'look', 'bad', 'time', 'film']

Topic 7
['scene', 'excellent', 'right', 'pretty', 'work', 'think', 'good', 'like', 'film', 'movie']

Topic 8
['character', 'good', 'really', 'suck', 'make', 'act', 'watch', 'movie', 'film', 'bad']

Topic 9
['suspense', 'special', 'look', 'story', 'film', 'character', 'interesting', 'love', 'like', 'movie']

Topic 10
['cast', 'bad', 'enjoy', 'think', 'time', 'character', 'good', 'great', 'film', 'movie']
```

Figure 4.3: Top words extracted for each topic using LDA

Task 4.5 Summary of Topic Detection

The topic modelling results demonstrated that LDA can successfully identify underlying themes in movie review text. This task complemented the sentiment analysis tasks by providing an additional layer of insight into the dataset beyond positive and negative classification.

Conclusion

This coursework presented a complete sentiment analysis pipeline applied to the IMDb labelled movie review dataset. The study began with thorough text preprocessing where noise such as punctuation, numbers and stopwords was removed and lemmatization was applied to standardise the text. These preprocessing steps were essential for improving the quality of the data and preparing it for further analysis.

Traditional machine learning approaches were then explored using the Bag of Words feature extraction technique. Three classifiers—Naïve Bayes, Logistic Regression and Support Vector Machine—were trained and evaluated.

The results showed that these models achieved reasonable performance; however, their reliance on word frequency limited their ability to capture contextual meaning within the text.

To overcome these limitations, a deep learning-based approach using BERT was implemented. The BERT model significantly outperformed the traditional models, achieving an accuracy of approximately 86.67%. This improvement highlights the advantage of transformer-based models in understanding contextual and semantic information in natural language.

In addition to sentiment classification, topic detection was performed using Latent Dirichlet Allocation (LDA). Topic modelling provided valuable insights into the underlying themes present in the movie reviews, such as movie quality, storyline and viewer experience. This task demonstrated how unsupervised learning techniques can complement sentiment analysis by revealing hidden patterns in text data.

Overall, the coursework demonstrated the effectiveness of combining traditional machine learning, deep learning, and unsupervised learning techniques for text analytics. The results clearly show that while traditional models are computationally efficient, deep learning models like BERT provide superior performance for sentiment analysis. Future work could involve experimenting with larger datasets, additional deep learning architectures or advanced topic coherence evaluation methods to further improve results.

This coursework also highlights the importance of selecting appropriate techniques based on the complexity of the task and available computational resources. While traditional models are faster and easier to implement, deep learning approaches provide superior performance when contextual understanding is required. The combination of multiple techniques in this study demonstrates a balanced and practical approach to text analytics.

References

1. Intelligent Data And Text Analytics Module

<https://moodle.port.ac.uk/course/view.php?id=2317>

2. Dataset

<https://archive.ics.uci.edu/dataset/331/sentiment+labelled+sentences>

4. Pandas

https://pandas.pydata.org/docs/user_guide/

5. Scikit Learn

https://scikit-learn.org/stable/user_guide.html

6. w3 schools

<https://www.w3schools.com/python/pandas/default.asp>